**Université du Québec à Chicoutimi**
Département des sciences appliquées
6GEI311 Architecture des logiciels

# Laboratoire 4

## Intégration du système de gestion de tickets par une API REST

Rapport

Par
**Équipe : Zongo T Julien, Zongo A Razack**

2026-11-11

Dépôt : github.com/abdoulrazackZONGO/Laboratoire4_6GEI311

# Table des matières

Introduction ..... 3
Section I. Modifications pour l'intégration par une interface REST ..... 4
I.1 Deux processus, cinq couches ..... 4
I.2 Conception de l'API ..... 4
I.3 Partie métier ..... 6
I.4 Le serveur ..... 6
I.5 Partie UI ..... 7
I.6 Bonus : la page web ..... 8
I.7 Tests et validation ..... 8
Section II. Leçons apprises ..... 9
Conclusion ..... 9
Annexe A. Lancer l'application et les tests ..... 10

# Introduction

Au Laboratoire 3, notre gestionnaire de tickets était une application Tkinter autonome. Les fenêtres appelaient directement les objets du modèle, et les données étaient stockées dans un fichier JSON sur le poste de l'utilisateur. Pour qu'un autre poste voie les mêmes tickets, il n'y avait pas de solution.

Au Laboratoire 4, nous avons coupé l'application en deux processus. Un serveur Flask détient le modèle et l'expose par une API REST. La GUI Tkinter n'appelle plus le modèle : elle envoie des requêtes HTTP à ce serveur, comme le fait aussi la page web que nous avons ajoutée en bonus. La figure 1 résume le changement.

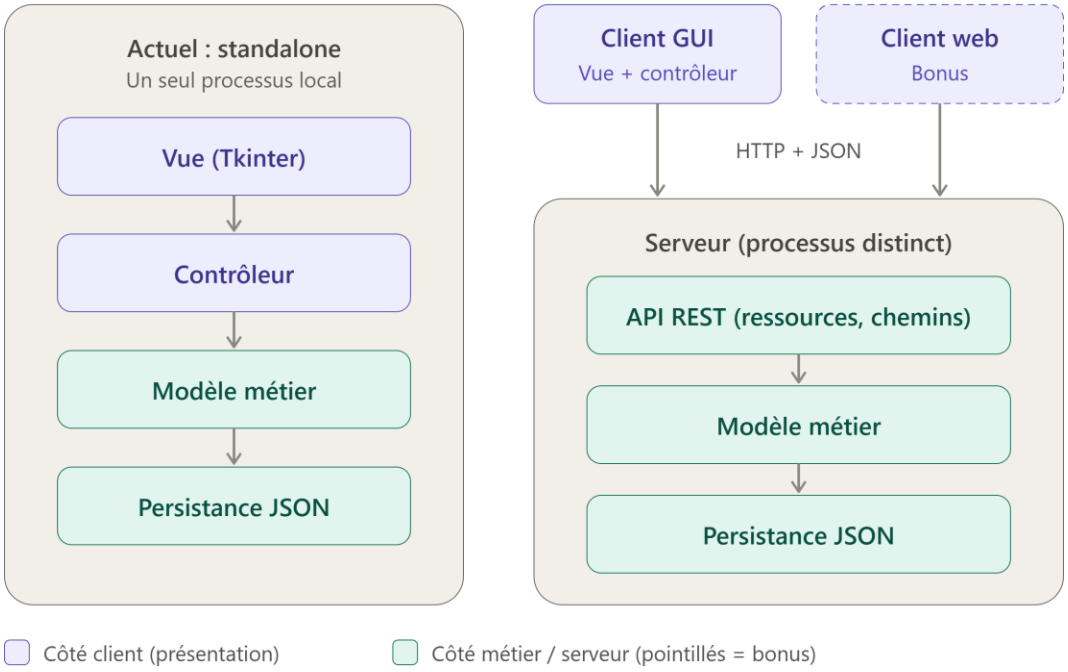



**Figure 1 : L'application du Lab 3 (à gauche) et celle du Lab 4 (à droite)**

La section I résume ce que nous avons modifié : d'abord la conception de l'API, puis la partie métier et le serveur, ensuite la partie UI et la page web, enfin les tests. La section II rassemble ce que ce laboratoire nous a appris. L'annexe A explique comment lancer le serveur, les clients et les tests.

# Section I. Modifications pour l'intégration par une interface REST

## I.1 Deux processus, cinq couches

Le tableau 1 donne les couches de l'application et ce que chacune a le droit de faire. Dans le vocabulaire MVC, le contrôleur est maintenant côté serveur : ce sont les routes de l'API. Les classes de ihm/controleurs restent dans la GUI ; pour éviter la confusion, nous les appelons logique de présentation.

**Tableau 1. Les couches et leurs règles.**

| Couche | Paquets | Rôle et règles |
|---|---|---|
| Modèle métier | modele, etats, acteurs, description, commandes, sessions | Porte toutes les règles. N'importe ni Flask, ni l'API, ni l'interface. |
| Persistance | persistance | Sauvegarde JSON (DepotJSON) et copies des images (StockageImages). |
| Contrôleur | api/ | Les routes Flask. Elles identifient l'acteur, vérifient la requête, appellent une méthode métier, sauvegardent et répondent en JSON. |
| Passerelle | ihm/passerelle | Le seul code du client qui fait du HTTP. |
| Présentation | ihm/controleurs, ihm/vues | Les fenêtres F1 à F8 et la logique qui les prépare. Seuls modele.enums, modele.exceptions et modele.observateur sont importés du métier. |

Ces règles sont vérifiées par un test, tests/test_regles_dependances.py, qui lit les imports de chaque fichier. Un import interdit fait échouer la suite, donc la séparation ne dépend pas de notre seule attention.

## I.2 Conception de l'API

Le fichier openapi.yaml décrit chaque chemin, ses corps de requête et de réponse, et ses codes de retour, avec un exemple par opération. Il a été relu par nous deux et fusionné avant la première route. Ensuite, Julien a écrit le serveur et Abdoul le client, chacun de son côté.

Nous avons identifié cinq ressources : l'acteur, le ticket (avec ses commentaires, sa description et ses images), le brouillon, le lot, et le compteur de version. Les chemins suivent la convention vue en cours : une collection au pluriel, une instance au singulier, et des paramètres explicites (/ticket/{ticketId}, et non /ticket/{id}). Le tableau 2 donne les 21 opérations.

**Tableau 2. Les opérations de l'API.**

| Méthode | Chemin | Usage |
|---|---|---|
| GET | /acteurs, /acteur/{acteurId} | Annuaire, lu avant toute connexion |
| GET | /tickets | Liste, filtrable par statut, créateur et assigné |
| POST | /tickets | Création en une seule requête (User) |
| GET | /ticket/{ticketId} et ses sous-chemins /commentaires, /description | Détail d'un ticket |
| PATCH | /ticket/{ticketId} | Changement de statut |

| Méthode | Chemin | Usage |
|---|---|---|
| DELETE | /ticket/{ticketId} | Suppression par le créateur d'un ticket jamais assigné |
| GET | /ticket/{ticketId}/image/{nomImage} | Octets d'une image de la description |
| GET, POST | /acteur/{acteurId}/brouillons | Brouillons ouverts, ouverture d'un brouillon |
| GET, PATCH, DELETE | /acteur/{acteurId}/brouillon/{brouillonId} | Lecture, pause ou reprise, abandon |
| GET, POST | .../brouillon/{brouillonId}/modifications | Modifications en attente, ajout |
| DELETE | .../brouillon/{brouillonId}/modification/derniere | Annuler le dernier ajout |
| POST | .../brouillon/{brouillonId}/application | Appliquer le brouillon |
| POST | /acteur/{acteurId}/lots | Lot de l'Admin, appliqué au mieux |
| GET | /version | Compteur de modifications du dépôt |

L'acteur est identifié par l'en-tête X-Acteur-Id. Comme au Lab 3, il n'y a pas de mot de passe : l'en-tête remplace le choix fait dans la fenêtre de connexion. Sans en-tête, ou avec un acteur inconnu, la réponse est 401.

Le changement de statut est un PATCH sur le ticket, et non une route par action. Le statut demandé choisit la méthode du modèle : ASSIGNE appelle Admin.assignTicket, OUVERT desassignerTicket, TERMINE validerTicket, FERME closeTicket, et VALIDATION User.soumettreValidation. La route ne vérifie jamais si la transition est permise ; c'est l'état du ticket qui l'accepte ou qui lève TransitionInvalideException.

Les brouillons ont été la décision la plus discutée. Au Lab 3, un ticket se crée ou s'édite dans une session qu'on peut mettre en pause, annuler pas à pas ou abandonner. Une création en une seule requête aurait fait perdre tout cela. Un brouillon est donc une session du modèle gardée par le serveur, avec un identifiant ; chaque ajout, la pause et l'application sont des requêtes. POST /tickets existe aussi, pour la page web et les essais.

Toutes les erreurs ont la même forme, {"erreur": {"type", "message"}}. Le type et le message reprennent ceux de l'exception métier, ce qui permet au client de relever exactement la même exception. Le tableau 3 donne la correspondance.

**Tableau 3. Exceptions et codes HTTP.**

| Origine | Code |
|---|---|
| TransitionInvalideException, SessionIndisponibleException, SuppressionInterditeException | 409 |
| AccesRefuseException, mauvais rôle, acteur du chemin différent de l'en-tête | 403 |
| ArtefactInvalideException, requête mal formée (JSON, champ, enum) | 400 |
| En-tête X-Acteur-Id absent, non entier ou inconnu | 401 |
| Ressource ou chemin introuvable | 404 |
| Méthode absente sur un chemin | 405 |
| Corps de plus de 8 Mo, image de plus de 5 Mo | 413 |

| Origine | Code |
|---|---|
| Toute autre exception : message générique, trace dans la console du serveur | 500 |

## I.3 Partie métier

Le modèle du Lab 3 a très peu changé. Le tableau 4 donne les seuls ajouts. La machine à états, les acteurs, les sessions, les commandes et DepotJSON sont restés tels quels, et la sortie de main.py est toujours celle du Lab 2 (test de non-régression).

**Tableau 4. Les ajouts à la partie métier.**

| Ajout | Pourquoi |
|---|---|
| SystemeTickets.ticketParId(id) | Une route reçoit un identifiant dans l'URL, et non un objet. |
| SystemeTickets.reserverIdentifiant() | C'est le serveur qui numérote les tickets, y compris ceux d'un brouillon qui n'est pas encore dans le dépôt. Un numéro ne sert qu'une fois tant que le serveur tourne. |
| Unicité des identifiants | ajouterTicket refuse un doublon avant toute notification, et DepotJSON refuse un fichier qui en contient. |
| StockageImages.enregistrer(nomFichier, octets) | Une image arrive en base64, et non comme un fichier du poste. Mêmes contrôles que copier(), et le nom reçu est ramené à un nom sûr pour le disque. |

L'API a fait apparaître quelques trous du modèle, que la GUI cachait en grisant ses boutons. Un commentaire vide est accepté par le modèle ; l'API le refuse comme une erreur de format. Une création sans texte ni image est acceptée, et un ticket FERME reste éditable. Enfin, soumettre à validation un ticket sans assigné donne 403 et non 409, parce que User.soumettreValidation vérifie l'assigné avant l'état. Nous les avons gardés tels quels et fixés par des tests, plutôt que d'ajouter des règles dans les routes.

## I.4 Le serveur

serveur.py lance Flask sur 127.0.0.1:5000. L'application est construite par une fabrique, creerApplication(contexte), ce qui permet aux tests de créer un serveur neuf sans réseau. Le ContexteServeur détient le SystemeTickets unique, le DepotJSON, le StockageImages et le registre des brouillons. Flask sert plusieurs requêtes à la fois : tout accès au système passe par un verrou (threading.RLock).

Chaque route suit les mêmes étapes, dans cet ordre :

- Identifier l'acteur par X-Acteur-Id (sinon 401) ;
- Vérifier le rôle et l'acteur du chemin (sinon 403), puis la forme du corps (sinon 400) ;
- Retrouver les objets par identifiant (sinon 404) ;
- Sous le verrou, appeler une méthode métier qui existe déjà ;
- Si quelque chose a changé, sauvegarder ;
- Convertir le résultat en JSON avec api/representations.py.

Les représentations sont distinctes de la sauvegarde. Un ticket expose toujours transitionsPermises et dejaAssigne, pour que les clients grisent leurs boutons sans recopier les règles. Le chemin disque d'une image n'est jamais exposé : seulement son URL. La route d'une image ne sert que les images de la description du ticket, et le nom reçu dans l'URL n'entre jamais dans un chemin de fichier.

Le registre des brouillons reprend côté serveur ce que ControleurEdition faisait dans la GUI au Lab 3. Il garde, pour chaque session, le texte tapé pendant la pause et une pile de copies d'images alignée sur celle de la session. Annuler le dernier ajout ou abandonner supprime ainsi la bonne copie.

Le patron Observateur ne traverse pas le réseau : une fenêtre de la GUI ne peut pas s'abonner à un objet qui vit dans un autre processus. Le serveur garde donc l'Observateur pour lui. Un petit observateur, abonné au SystemeTickets, compte les modifications, et GET /version rend ce compteur. Les clients l'interrogent chaque seconde et ne relisent les tickets que s'il a changé.

## I.5 Partie UI

Côté client, le travail a consisté à remplacer le modèle par l'API sans réécrire les fenêtres. Le tableau 5 résume les changements.

**Tableau 5. Les changements de la partie UI.**

| Élément | Changement |
|---|---|
| ihm/passerelle/client_api.py (nouveau) | ClientAPI(url, acteurId) : une méthode par opération du contrat, urllib, délai de 2 s. Une réponse d'erreur relève l'exception métier de même nom et de même message ; un serveur muet lève ServeurInjoignable. |
| ihm/passerelle/objets_lecture.py (nouveau) | TicketLu, ActeurLu, BrouillonLu, RapportLu... Ils gardent les noms de propriétés du modèle (ticketID, title, status, creator.name, transitionPossible), donc les vues ont peu changé. Ils se comparent par identifiant. |
| ihm/passerelle/source_tickets.py (nouveau) | SourceTickets, copie locale observable. Elle sonde /version, compare les tickets reçus et notifie ajouts, retraits et modifications par les méthodes de l'Observateur du Lab 3. |
| ControleurPrincipal | Ne détient plus ni données ni sauvegarde. Chaque session ouverte depuis F1 a sa ClientAPI et sa SourceTickets ; une seule boucle after() les sonde. |
| ControleurAdmin, ControleurUser | Listes par GET /tickets et ses filtres, boutons d'après transitionsPermises, lots par POST /lots, suppression et soumission par l'API. |
| ControleurEdition | Passe par les routes du brouillon. Les images sont lues sur le poste puis envoyées en base64. |
| ControleurDetail (nouveau) | F6 télécharge les images par l'API et les affiche avec tk.PhotoImage(data=...). Une image illisible est signalée sans bloquer le reste. |
| FenetreBase | executer() affiche aussi ServeurInjoignable. Une seule boîte d'erreur s'affiche si le serveur s'arrête, et les listes reprennent quand il revient. |

Deux détails ont demandé de l'attention. Chaque rafraîchissement crée de nouveaux objets TicketLu : comparer deux tickets avec is, comme au Lab 3, échouait toujours. Toutes les comparaisons se font

maintenant par identifiant. Ensuite, un brouillon n'est pas dans le dépôt, donc /version ne bouge pas quand on le met en pause. Après une action dans F4 ou F5, le contrôleur principal prévient lui-même les fenêtres F3 ouvertes, qui relisent la liste des brouillons.

Les contrôleurs sont testés contre les vraies routes : tests/serveur_essai.py branche la passerelle sur le client de test de Flask, sur les données de démonstration et sans réseau.

## I.6 Bonus : la page web

Le serveur sert aussi une page web (web/index.html, app.js et style.css), sans framework ni étape de compilation. Elle ne passe que par l'API, comme la GUI. Un Admin y voit tous les tickets et change leur statut ; un User y voit les onglets « Créés par moi » et « Assignés à moi », crée un ticket, le soumet à validation ou le supprime. La page interroge /version chaque seconde : un ticket créé dans la page apparaît dans la GUI en une ou deux secondes, et l'inverse.

Les données y sont toujours insérées comme du texte, jamais comme du HTML : un titre de ticket ne peut pas injecter de script. Chaque requête a un délai de 5 s, et un bandeau prévient quand le serveur ne répond plus.

## I.7 Tests et validation

La suite compte 568 tests, lancés par python -m unittest discover -s tests -t . et par GitHub Actions à chaque push. Le tableau 6 les regroupe.

**Tableau 6. Les tests par partie.**

| Partie | Ce qui est vérifié | Tests |
|---|---|---|
| Serveur | Routes de lecture et d'écriture, brouillons et leur registre, lots, démarrage, validation des corps | 251 |
| Robustesse | Une ligne de la table des erreurs par test, requêtes simultanées, contrat openapi.yaml comparé aux routes, outils/essai_api.py | 36 |
| Partie métier | Modèle des labs 2 et 3, identifiants, persistance, images, non-régression de main.py | 122 |
| Partie UI | Passerelle, source observable, logique de présentation, règles de dépendances | 147 |
| Page web et dépôt | Fichiers servis, appels du script, absence de tiret long dans le dépôt | 12 |

Avant la démonstration, outils/essai_api.py enchaîne quelques requêtes réelles contre le serveur lancé : lecture, création, assignation, transition refusée et lot. Il affiche chaque code reçu à côté du code attendu. Nous avons aussi déroulé à la main un scénario de cinq minutes avec un serveur, deux GUI (un Admin, un User) et une console PowerShell.

## Section II. Leçons apprises

Figer le contrat d'abord, encore une fois. Au Lab 3, figer les signatures des vues nous avait permis de travailler en parallèle. Ici, openapi.yaml a joué le même rôle à plus grande échelle. Le serveur et le client ont avancé chacun sur leurs fichiers, et la passerelle a même été testée contre les exemples du contrat avant que les routes existent. L'intégration n'a révélé qu'un écart entre les deux côtés : l'affichage des accents dans PowerShell.

Un Observateur ne traverse pas le réseau. Nous pensions garder le patron tel quel. En pratique, la GUI ne peut pas s'abonner à un objet d'un autre processus. Le compteur /version, sondé chaque seconde, est simple et suffit pour une démonstration, mais il a un coût : une requête par seconde et par session ouverte, et jusqu'à une seconde de retard. Il ne voit pas non plus les brouillons, qui ne sont pas dans le dépôt. Avec plus de temps, nous essaierions un canal où le serveur pousse les changements.

Un test de concurrence qui passe ne prouve pas grand-chose. Notre premier test lançait des créations simultanées et vérifiait que les identifiants étaient tous différents. Il passait encore quand on retirait le verrou, parce que l'interpréteur Python rend ces courses très rares. Nous avons dû mesurer directement combien d'appels se croisaient. Sans verrou, le test en voit maintenant trois ou quatre à la fois.

L'API montre les trous du modèle. Tant que la GUI grisait ses boutons, personne ne pouvait envoyer un commentaire vide ou soumettre un ticket sans assigné. Une API accepte n'importe quelle requête, donc chaque règle absente du modèle devient visible. Nous avons choisi de ne pas les recopier dans les routes, et de les noter ici. Elles devraient rejoindre le modèle.

Quelques limites restent assumées. L'en-tête X-Acteur-Id n'est pas une authentification. Les brouillons ne sont pas sauvegardés : un redémarrage du serveur les perd, et laisse leurs copies d'images dans donnees/images/. Le serveur réécrit tout le fichier JSON à chaque modification, ce qui convient à quelques dizaines de tickets mais pas à beaucoup plus.

Pour l'organisation, nous avons découpé le travail en tâches courtes, une branche par tâche (T0, S1 à S5 pour le serveur, C1 à C6 pour le client, I1 pour l'intégration, W1 pour la page web), et chaque branche est passée par une pull request. Couper par couche, le long de la frontière HTTP, a évité presque tous les conflits de fusion.

## Conclusion

Le système fonctionne maintenant en client-serveur. Le modèle des laboratoires 2 et 3 n'a reçu que quatre ajouts, et toutes ses règles sont restées dans les états et les acteurs. La GUI a gardé ses huit fenêtres et ses tactiques d'utilisabilité, y compris la pause et l'annulation pas à pas, grâce aux brouillons. La page web montre qu'un deuxième client s'ajoute sans toucher au serveur, sauf la route qui la sert.

# Annexe A. Lancer l'application et les tests

Depuis le dossier code, après pip install -r requirements.txt :

- Serveur : python serveur.py (http://127.0.0.1:5000).
- GUI : python app.py, dans un autre terminal, autant de fois que de fenêtres voulues.
- Page web : ouvrir http://127.0.0.1:5000/ dans un navigateur.
- Vérification rapide avant une démonstration : python outils/essai_api.py.
- Tests : python -m unittest discover -s tests -t .
- Repartir de la démonstration : arrêter le serveur et supprimer donnees/sauvegarde.json.